

The problem: Conversation history alone isn't enough for programmatic control

Introduction

You've learned to build agents with sophisticated instructions. Your agents can respond intelligently using conversation history—the LLM automatically receives previous messages and understands context. But what if **your code** needs to check specific values or make programmatic decisions based on conversation data?

For example:

- How do you check “Has the user provided their name?” in your code?
- How do you store an extracted value for later use in your application?
- How do you make routing decisions based on exact data (not LLM interpretation)?

The limitation

Conversation history is sent to the LLM, but your code cannot programmatically access it. You can't write `if conversation_history.contains("Alex")`: because conversation history isn't a data structure your code can query—it's just text messages sent to the model.

This part introduces **session state**—a dictionary your code can read and write programmatically.



The problem:

Conversation history alone isn't enough for programmatic control

Let's see what happens when you try to build agents that need to make decisions based on conversation data:

```
Python
from google.adk.agents import LlmAgent

# Agent that should extract and remember user's name
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Extract the user's name and remember it for future questions."
)

# Turn 1
# User: "My name is Alex"
# Agent: "Nice to meet you, Alex!"

# Turn 2
# User: "What's my name?"
# Agent: "Your name is Alex" ✓ (LLM remembers from history)

# But in your code:
# How do you check if name was provided? ✗
# How do you access the exact name "Alex"? ✗
# How do you use it in application logic? ✗
```

The root problem

While the LLM can read conversation history and respond appropriately, your code cannot programmatically check values or make routing decisions. You need structured, programmatically accessible data.

